

关键路径

我们在写拓扑排序的时候提到，拓扑排序是关键路径的基础，那么为什么这么说呢？在此之前我们先看一些简单但很重要的概念。

基本概念

什么是AOE网？

AOE网（Activity On Edge）即边表示活动的网，是与AOV网（顶点表示活动）相对应的一个概念。而拓扑排序恰恰就是在AOV网上进行的，这是拓扑排序与关键路径最直观的联系。AOE网是一个带权的有向无环图，其中顶点表示事件（Event），弧表示活动，权表示活动持续的时间。下面的就是一个AOE网：

其中V1, V2, V3.....V9表示事件，a1.....a11表示活动，活动的取值表示完成该活动所需要的时间，如a1 = 6表示完成活动a1所需要的时间为6天。此外，每一事件Vi表示在它之前的活动已经完成，在它之后的活动可以开始，如V5表示活动a4和a5已经完成，活动a7和a8可以开始了。

AOE网的源点和汇点？

由于一个工程中只有一个开始点和一个完成点，故将AOE网中入度为零的点称为源点，将出度为零的点称为汇点。

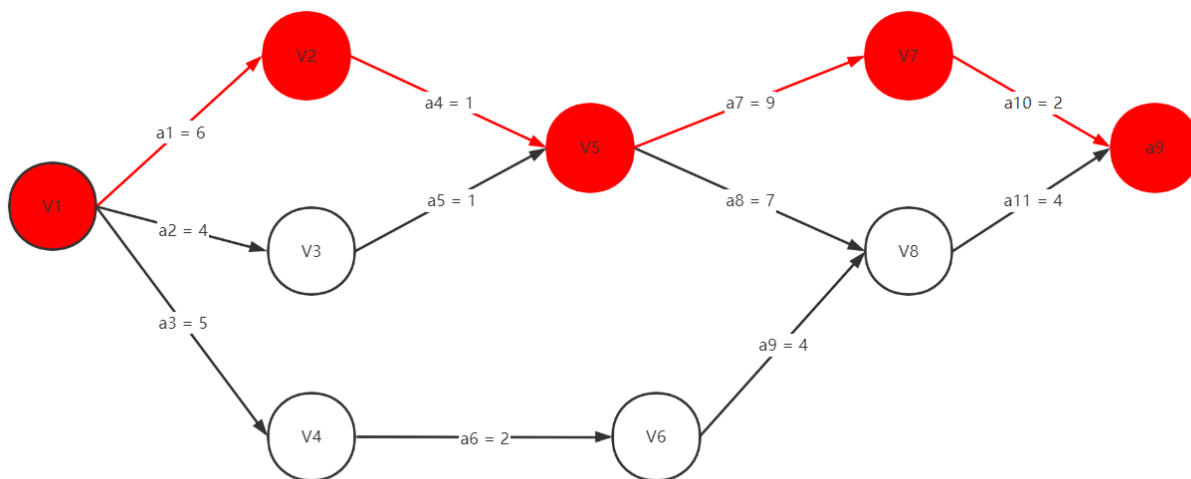
打个比方，我们现在有一个工程，就是将大象装进冰箱，那么源点就相当于我们现在接到这样一个任务，而汇点则表示我们完成了这个任务。那么我们之前所讲的打开冰箱门，将大象装进去，关上冰箱门就属于活动本身（即a1....a11所表示的信息），打开冰箱门所需要的时间就是活动所需要的时间，而完成某一个活动所到达的顶点就表示一个事件（冰箱门打开）。下图中的红色顶点V1表示源点，V9表示汇点。

什么是关键路径呢？

举个栗子。

唐僧师徒从长安出发去西天取经，佛祖规定只有四人一起到达西天方能取得真经。假如师徒四人分别从长安出发，走不同的路去西天：孙悟空一个筋斗云十万八千里，一盏茶的功夫就到了；八戒和尚稍慢点也就一天左右的时间；而唐僧最慢需要14年左右。徒弟到达后是要等着师傅的。那么用时最长的唐僧所走的路，就是取经任务中的关键路径。其他人走的路径属于非关键路径。

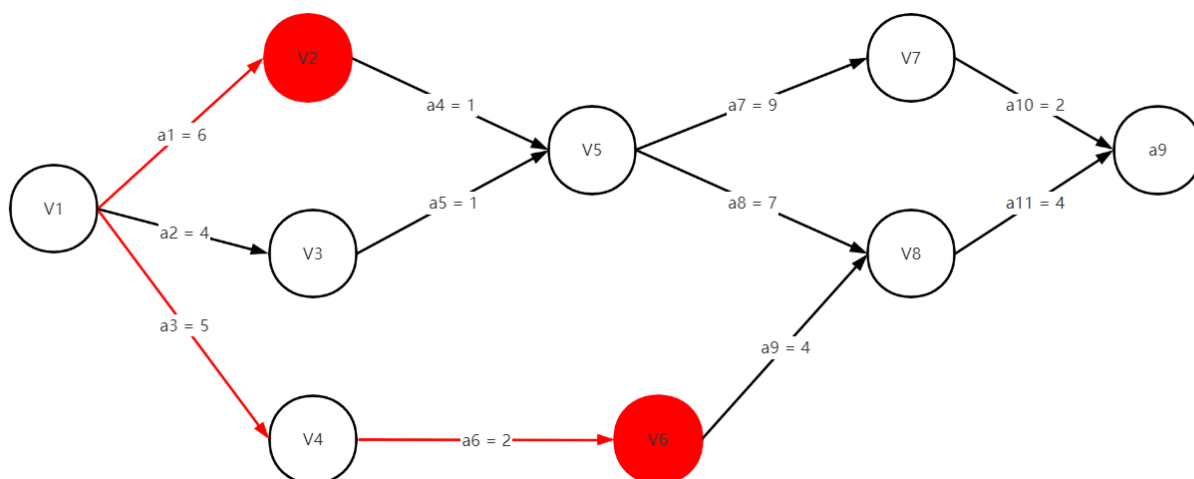
由于AOE网中的有些活动是可以并行进行的（如活动a1、a2和a3就是可以并行进行的），所以完成工程的最短时间是从源点到汇点的最长路径的长度。路径长度最长的路径就叫做 **关键路径（Critical Path）**。如下图红色顶点和有向边构成的就是一条关键路径，关键路径的长度就是完成活动 a1、a4和a9、a10所需要的时间总和，即为 $6+1+9+2 = 18$ 。



我们现在对于关键路径、源点、汇点、事件、活动和AOE网有了一个清晰的认识，但我们要想在一个有向图当中找到这样一条关键路径还得清楚下面的四个概念。

什么是ETV?

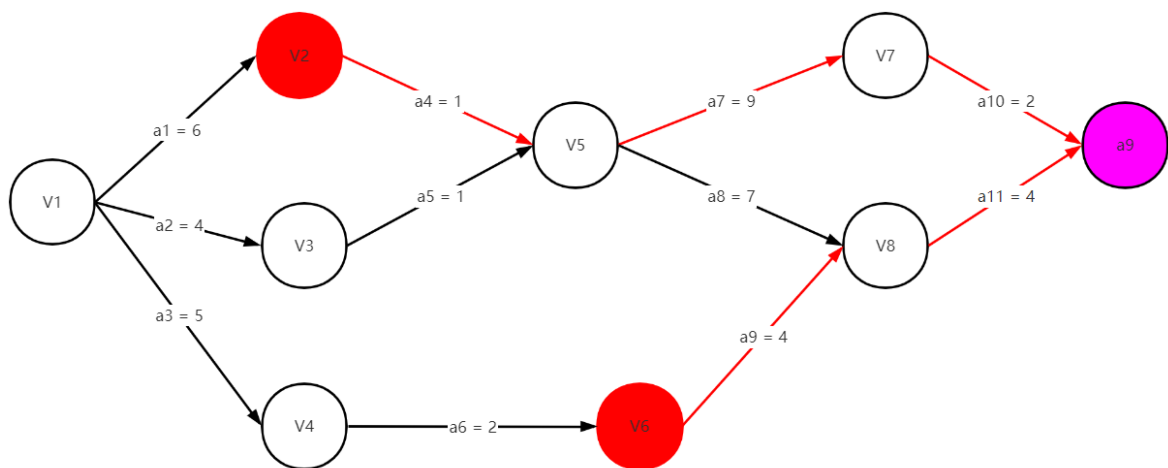
ETV (Earliest Time Of Vertex): 事件最早发生时间，就是顶点的最早发生时间；(看图说话)



事件V2的最早发生时间表示从源点V1出发到达顶点V2经过的路径上的权值之和，从源点V1出发到达顶点V2只经过了权值为6的边，则V2的最早发生时间为6，表示在活动a1完成之后，事件V2才可以开始；同理，事件V6要发生（即最早发生）需要活动a3和活动a6完成之后才可以，故事件V6的最早发生时间为 $5 + 2 = 7$ 。其他顶点（事件）的最早发生时间同理可的。需要说明，事件的最早发生时间一定是从源点到该顶点进行计算的。

什么是LTV?

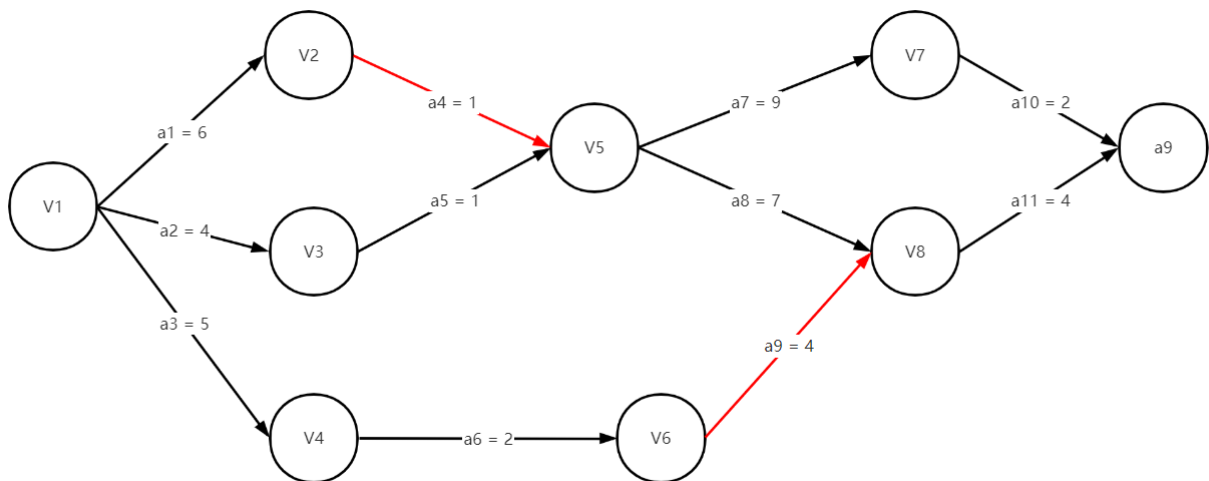
LTV (Latest Time Of Vertex): 事件最晚发生时间，就是每个顶点对应的事件最晚需要开始的时间，如果超出此时间将会延误整个工期。



前面在谈关键路径的概念时给出了一条上图中的关键路径，该关键路径（V1，V2，V5，V7，V9）的长度为18，为什么要提这个长度呢，因为要计算某一个事件的最晚发生时间，我们需要从汇点V9进行倒推。计算顶点V2的最晚发生时间为例，已知关键路径的长度为18，事件V2到汇点V9所需要的时间为 $1 + 9 + 2 = 12$ ，则事件V2的最晚发生时间为 $18 - 12 = 6$ ，这时候我们发现，这和事件V2的最早发生时间不是一样吗？的确如此，对于关键路径上的顶点都满足最早发生时间 etv 等于 最晚发生时间 ltv 的情况，这也是我们识别关键活动的关键所在。再来计算一下事件V6的最晚发生时间，事件V6到汇点V9所需要的时间为 $4 + 4 = 8$ ，则事件V6的最晚发生时间为 $18 - 8 = 10$ ；相当于说活动a6完成之后，大可以休息3天，再去完成活动a9也不会影响整个工期。

什么是ETE?

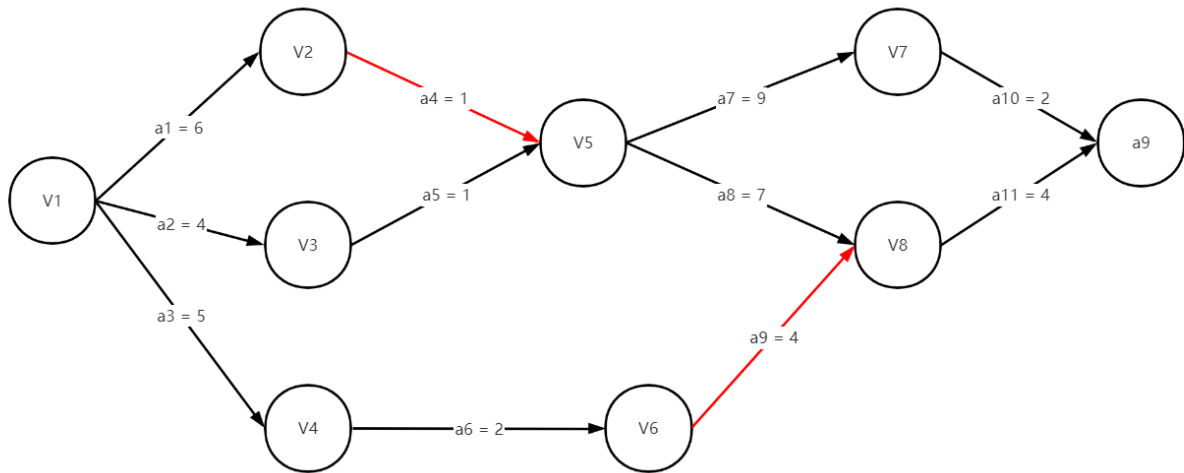
ETE(Earliest Time Of Edge)：活动的最早开工时间，就是弧的最早发生时间。



活动a4要最早开工时间为事件V2的最早发生时间 6；同理，活动a9的最早开工时间为事件v6的最早发生时间 7。显然活动的最早开工时间就是活动发生前的事件的最早开始时间。

什么是LTE?

LTE(Lastest Time of Edge): 活动的最晚开工时间, 就是不推迟工期的最晚开工时间。



活动的最晚开工时间则是基于事件的最晚发生时间。比如活动a4的最晚开工时间为事件V5的最晚发生时间减去完成活动a4所需时间, 即 $7 - 1 = 6$; 活动 a9的最晚开工时间为事件V8的最晚发生时间减去完成活动a9所需时间, 即 $14 - 4 = 10$;

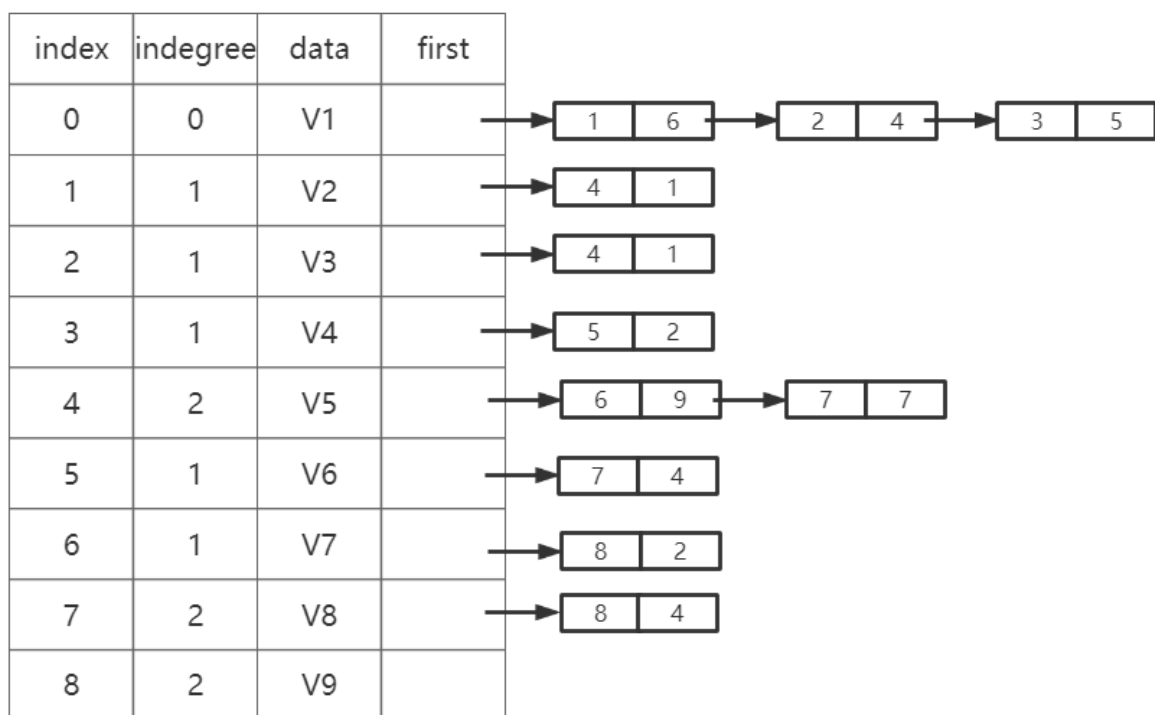
从上面也就可以看出 只要知道了每一个事件(顶点)的ETV和LTV, 就可以推断出对应的ETE和LTE。此外还需要注意, 关键路径是活动的集合, 而不是事件的集合, 所以当我们求得ETV和LTV之后, 还需要计算ETE和LTE。

关键路径算法

求关键路径的过程事实上最重要的就是上面提到的四个概念, ETV、LTV、ETE和LTE, 求得了ETE与LTE之后只需要判断两者是否相等, 如果相等则为关键路径中的一条边, 则输出。为了清晰的明白算法的执行流程, 我们一起 Step By Step.

首先求事件的最早发生事件ETV, 这里使用的就是之前分享的拓扑排序, 我们也走一遍, 算是对拓扑排序的回顾。

第一步, 建立图所对应的邻接表。



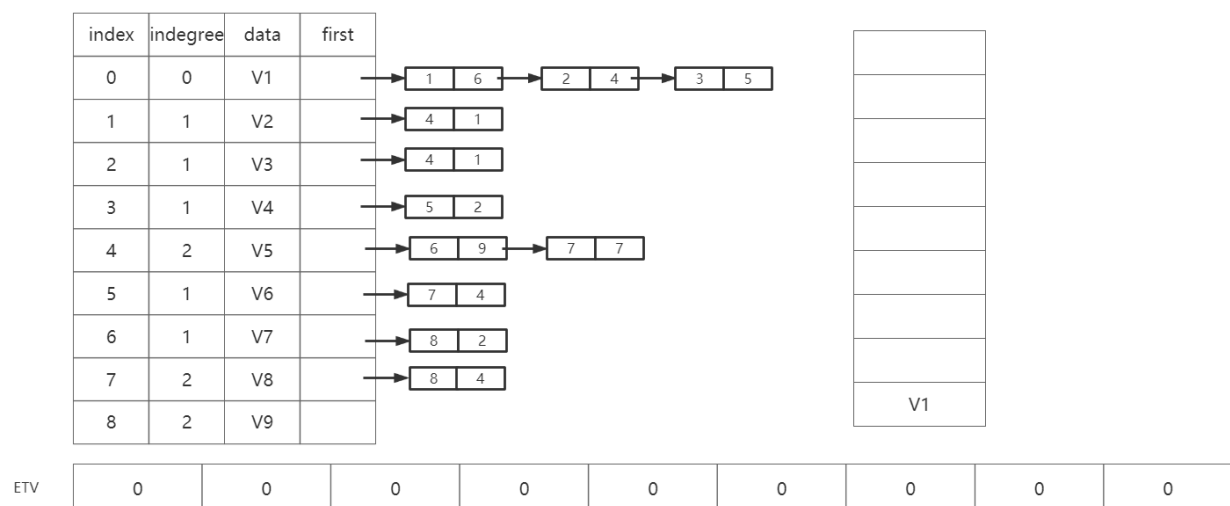
拓扑排序获得每一个事件的最早发生时间

初始时将ETV当中的数据都设置为0：

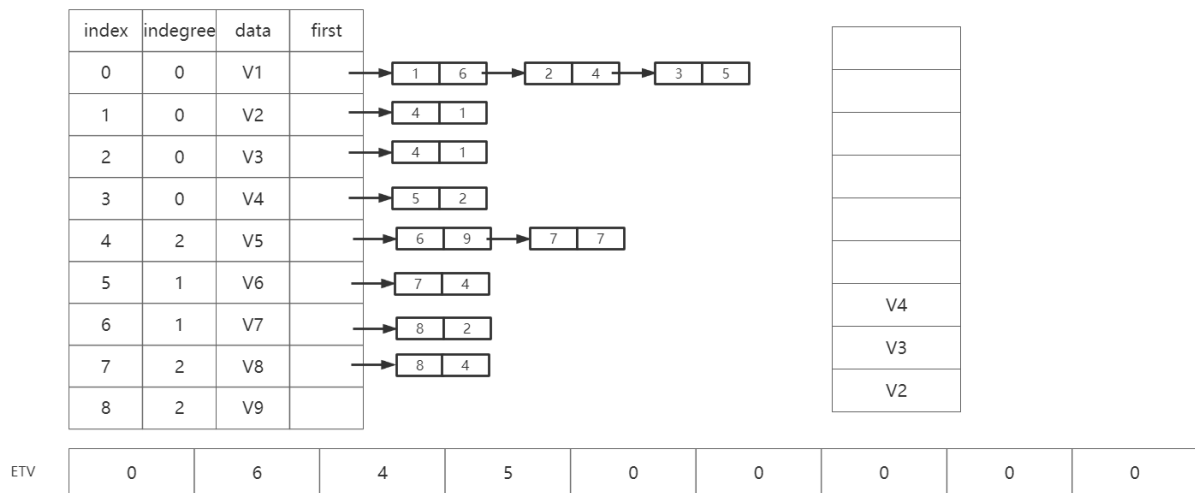
ETV	0	0	0	0	0	0	0	0
-----	---	---	---	---	---	---	---	---

下面开始便是拓扑排序的过程了。

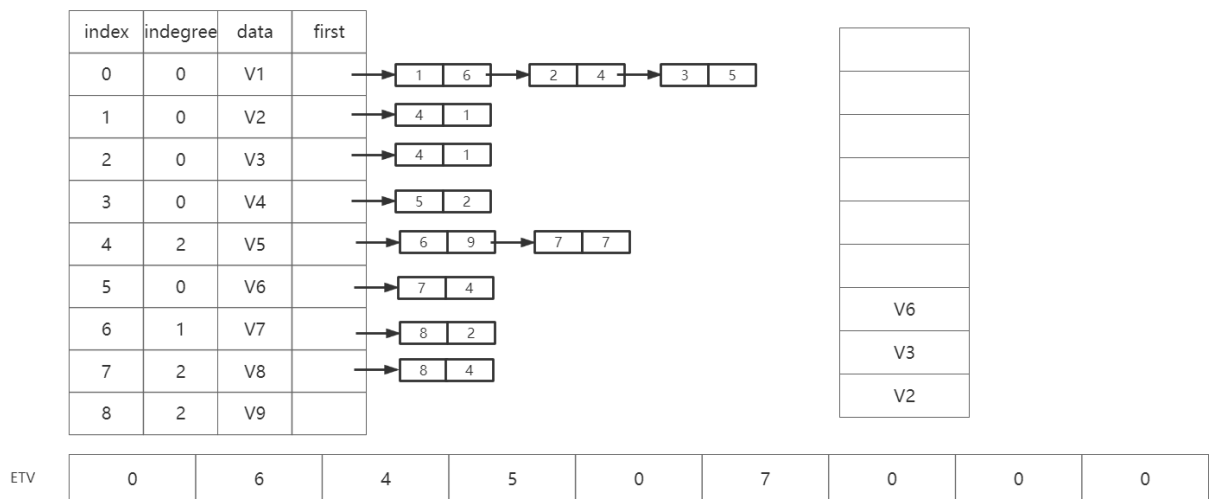
第二步：将入度为零的顶点V1入栈；



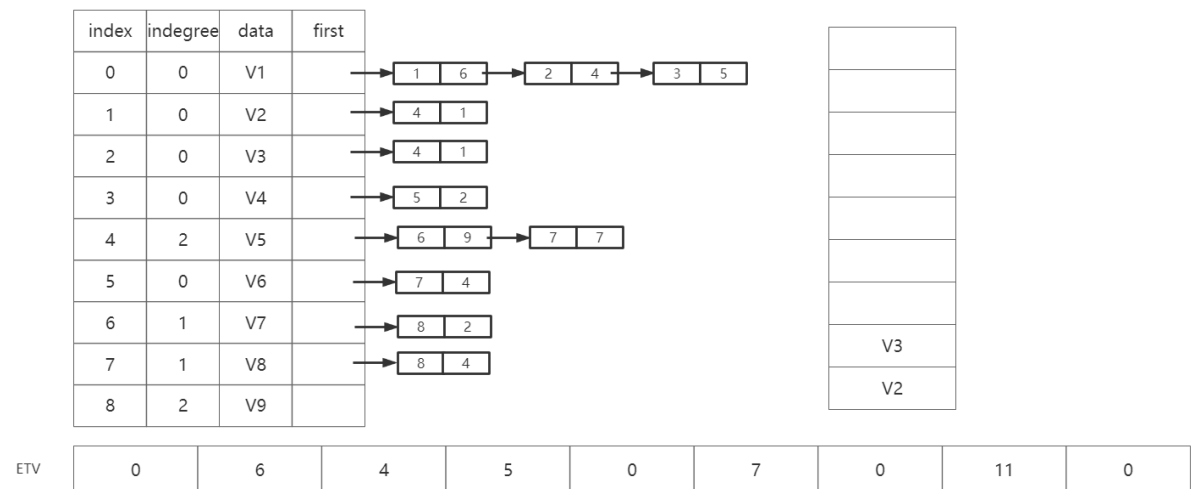
第三步：顶点V1出栈，并遍历顶点V1的邻接顶点，即 index 分别为 1, 2, 3 的顶点V2、V3和顶点V4，并将邻接顶点的入度减1，判断是否为零，如果为零则入栈。首先遍历 index = 1 顶点V2，并将顶点V2的入度减1，发现为零，则将顶点V2入栈，判断 $ETV[0] + w(V1, V2)$ 是否大于 $ETV[1]$ ，发现大于，则将 $ETV[1]$ 更新为 $ETV[0] + w(V1, V2) = 6$ ；同理遍历 index = 2 的顶点 V3，将顶点V3入栈，并将 $ETV[2]$ 更新为 4；遍历 index = 3 的顶点 V4，将顶点 V4入栈，并将 $ETV[3]$ 更新为 5。



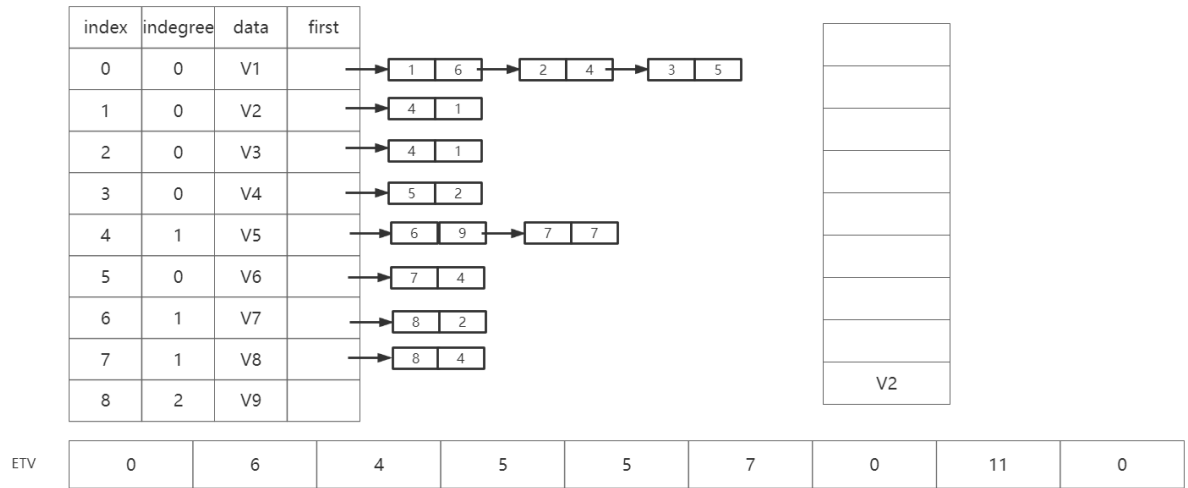
第四步：弹出栈顶元素V4并遍历顶点V4的邻接顶点V6. 将顶点V6的入度减一，并将 ETV[5] 更新为 $ETV[3] + w(V4,V6)$ ，即为7.



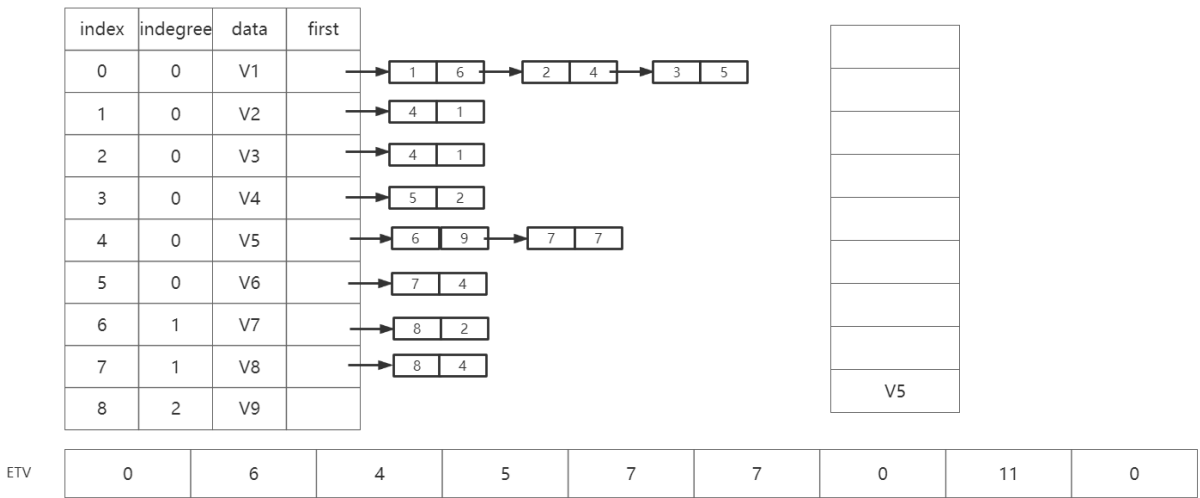
第五步：弹出栈顶元素 V6 并遍历邻接顶点V8 .



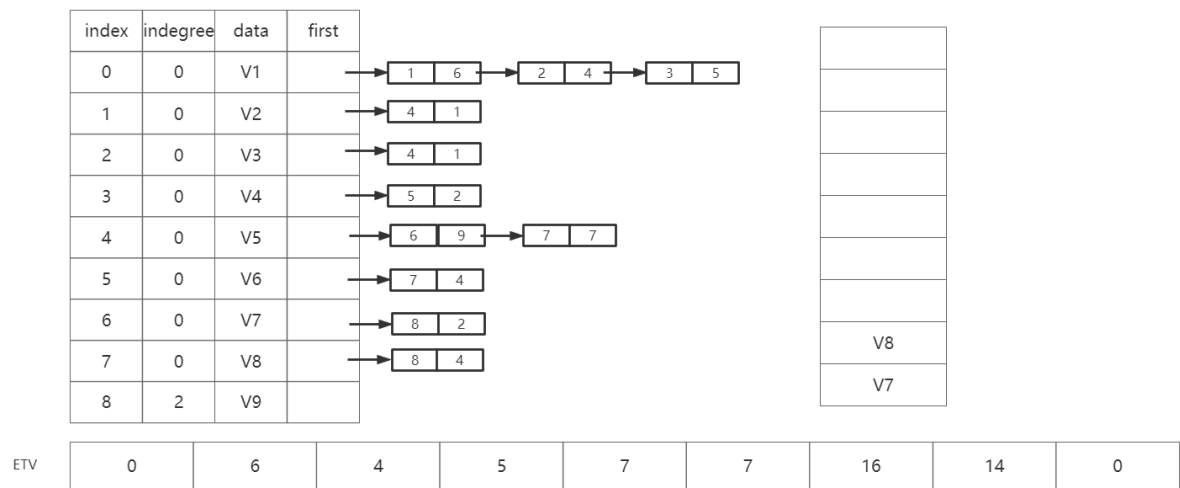
第六步：弹出栈顶元素 V3，并遍历V3 的邻接顶点V5 .



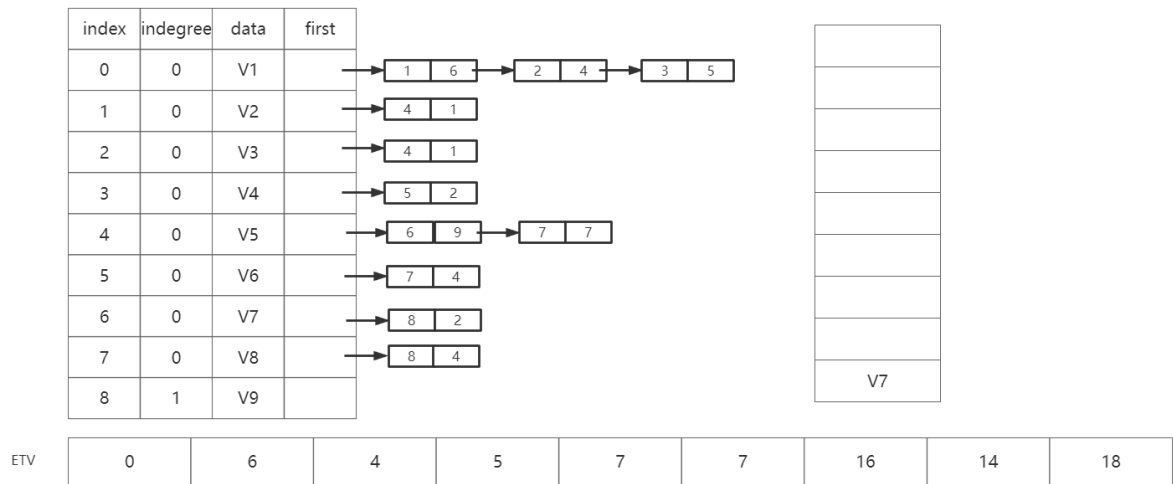
第七步：弹出栈顶元素V2,并遍历V2的邻接顶点V5



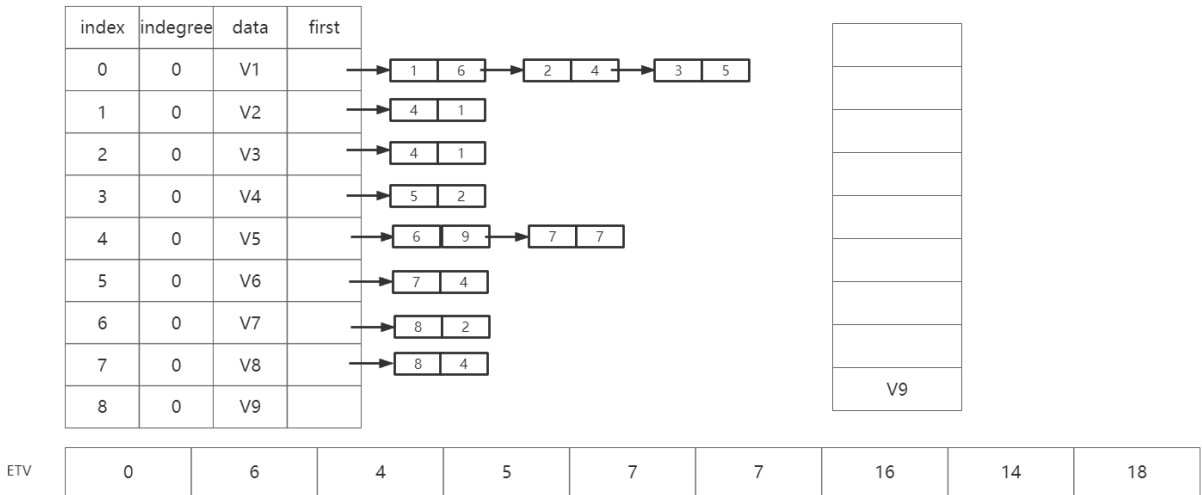
第八步：弹出栈顶元素V5 ,并遍历V5的邻接顶点V7和V8.



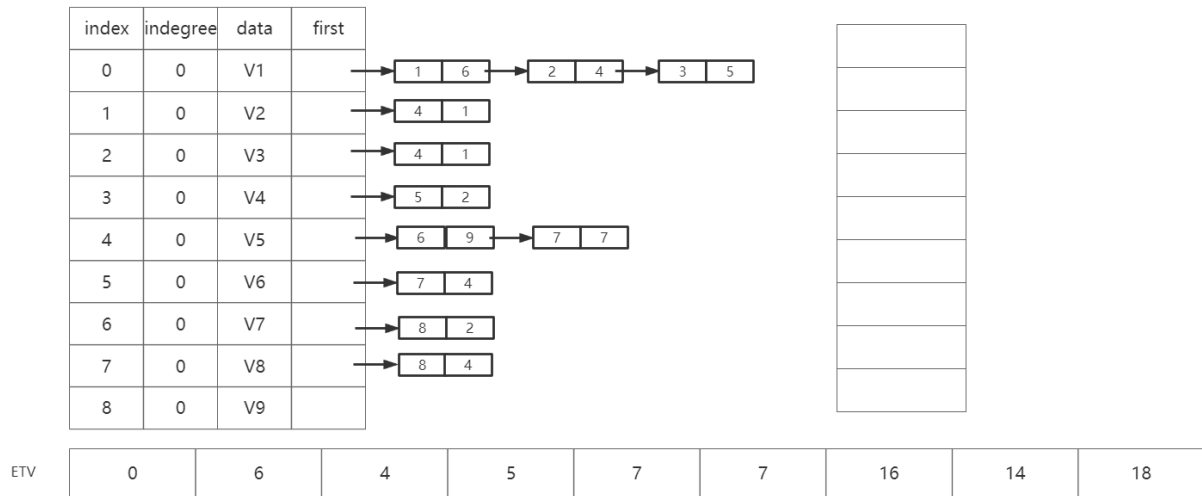
第九步：弹出栈顶元素 V8,并遍历V8的邻接顶点V9 。



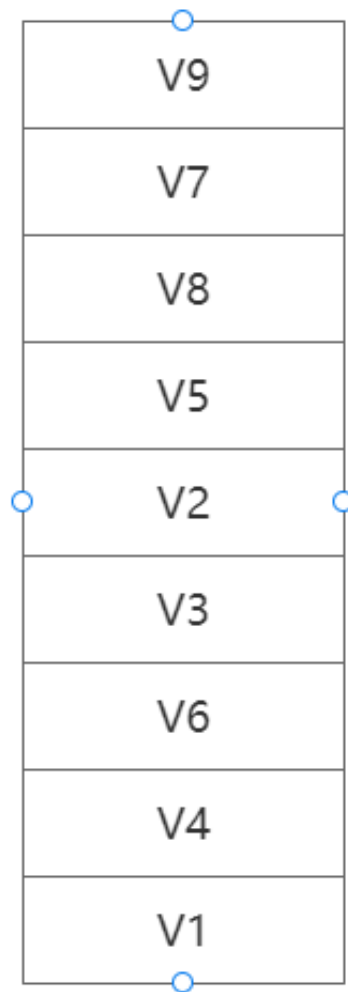
第十步：弹出栈顶元素V7 ,并遍历V7的邻接顶点V9 。



第十一步：将栈顶元素V9出栈。



其中，我们将拓扑排序过程中的出栈序列依次入栈，即拓扑排序结束时，我们响应的保存了一个出栈序列V1 V4 V6 V3 V2 V5 V8 V7 V9，用于逆向计算事件的最晚发生时间：



根据事件的最早发生时间ETV推断事件的最晚发生时间 LTV

通过拓扑排序我们获得了每一个顶点的最早发生时间 ETV，紧接着便可以计算 LTV了。

计算LTV的值，即每一个顶点的最晚发生时间值，初始时将LTV的值初始化为汇点V9的时间 18 。

LTV	18	18	18	18	18	18	18	18	18
-----	----	----	----	----	----	----	----	----	----

紧接着则是访问之前的出栈序列栈 。

首先将顶点V9出栈，什么都不做。

LTV	18	18	18	18	18	18	18	18	18
-----	----	----	----	----	----	----	----	----	----

V7
V8
V5
V2
V3
V6
V4
V1

第二步：将顶点V7出栈，并判断LTV[6] 与 $LTV[8] - w(V7,V9)$ 的大小，将 LTV 更新为 $LTV[8] - w(V7,V9) = 16$

LTV	18	18	18	18	18	18	16	18	18
-----	----	----	----	----	----	----	----	----	----

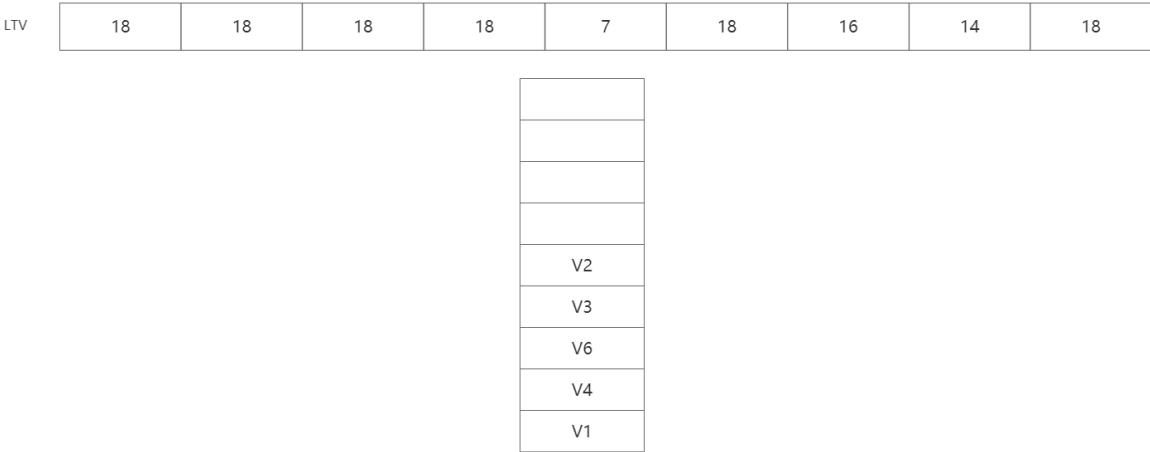
V8
V5
V2
V3
V6
V4
V1

第三步：将顶点V8出栈，更新 LTV[7] 为 $LTV[8] - w(V8,V9)= 14$ 。

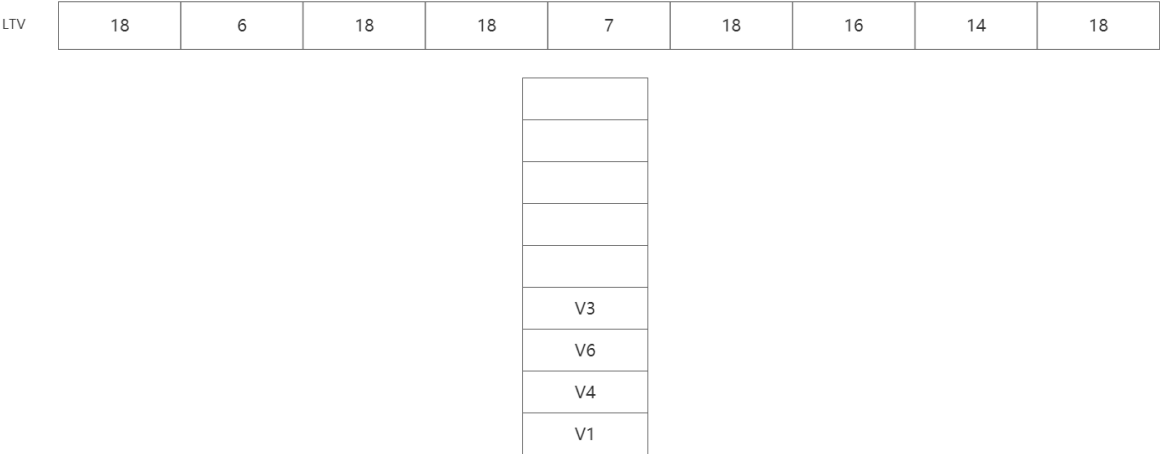
LTV	18	18	18	18	18	18	16	14	18
-----	----	----	----	----	----	----	----	----	----

V5
V2
V3
V6
V4
V1

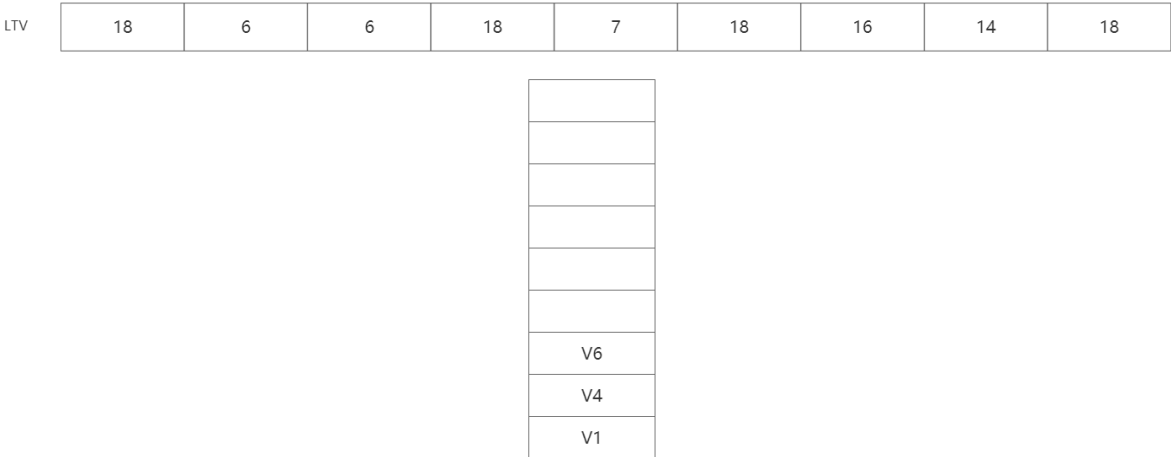
第三步：将顶点V5 出栈，并遍历V5的邻接顶点V7和V8。用邻接顶点V7的LTV值减去 活动a7的值，并与顶点V5的LTV值进行比较，将LTV[4] 更新为 7；同理用邻接顶点V8的LTV值减去活动a8的值，并与V5顶点的值进行比较，发现相等不做更新。



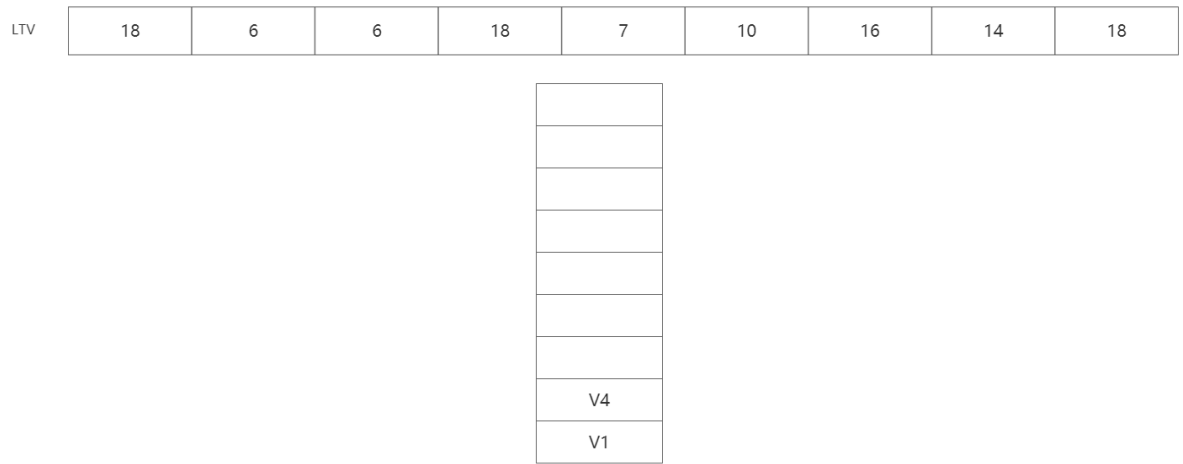
第四步，弹出栈顶元素V2 ,并遍历 V2的邻接顶点V5 ,用顶点V5的LTV(最晚发生时间) 减去活动a4所需时间，即为6，将顶点V2的LTV[1] 更新为6.



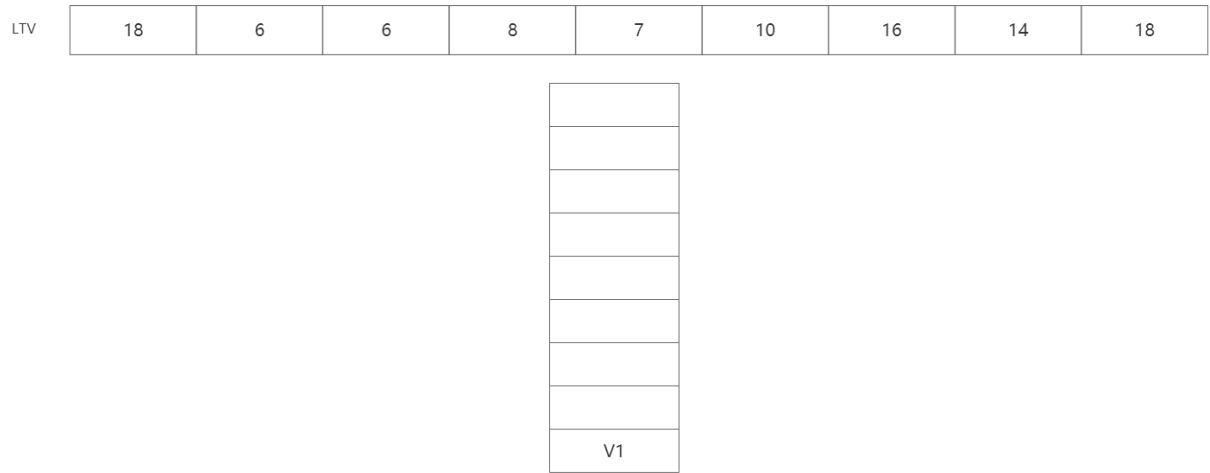
第五步，弹出栈顶事件（顶点）V3 ,并更新顶点V3的最晚发生时间。



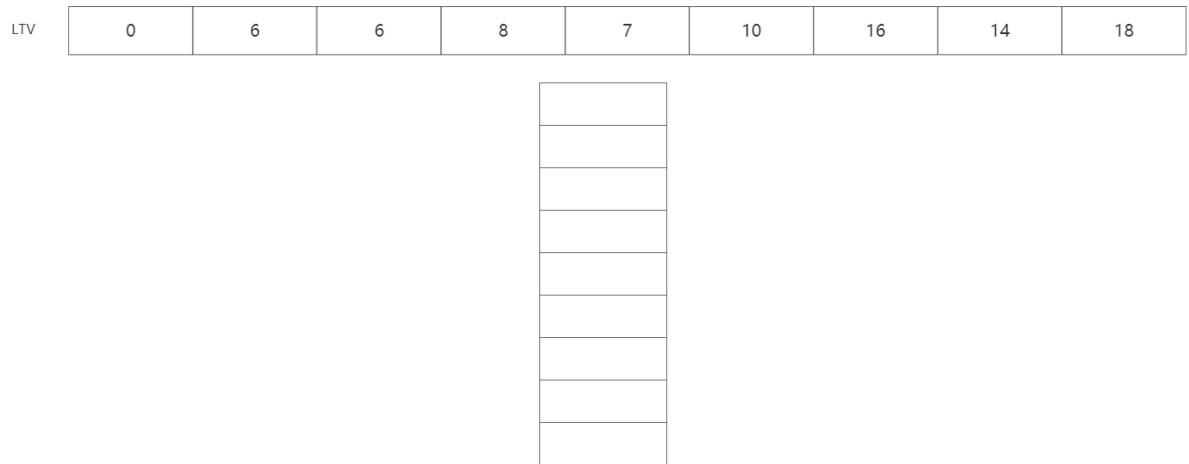
第六步：弹出栈顶事件V6（顶点） .



第七步：弹出栈顶元素V4并更新其最晚发生时间。



第八步：弹出栈顶元素V1并更新其最晚发生时间。使用事件V2的最晚发生时间 6 减去活动a1所需时间6 等于0，与顶点V1的最晚时间相比较并更新为0。需要说明，事实上源点 的最晚发生时间与最早发生时间都为0，毋庸置疑。



此时我们得到每个事件（顶点）的最早发生时间和最晚发生时间，但 关键路径是活动的集合，所以接下来我们用事件的最早发生时间和最晚发生时间计算出活动（弧）的最早与最晚发生时间即可

ETV	0	6	4	5	7	7	16	14	18
LTV	0	6	6	8	7	10	16	14	18

计算活动的最早与最晚开工时间

第一步：从源点V1开始，遍历源点的邻接顶点 V2，将弧 <V1,V2> 上的活动a1 的最早发生时间更新为源点 V1的最早发生时间 0，将活动a1 的最晚发生时间更新为事件V2的最晚发生时间 6 减去 活动 a1 所需要的时间6，即为0. 判断活动a1的最早发生时间与最晚发生时间是否相等，如果相等则为关键活动，并输出。同理遍历源点v1的邻接顶点v3和v4，并更新弧a2和a3的最晚开工时间与最早开工时间。

ETV	0	6	4	5	7	7	16	14	18
LTV	0	6	6	8	7	10	16	14	18
ETE	0	0	0						
LTE	0	2	3						

顶点	顶点	权值
V1	V2	6

第二步：访问顶点 V2，遍历V2的邻接顶点V5 . 将活动a4的最早发生时间更新为 事件 V2的最早发生时间，将活动 a4的最晚发生时间更新为事件 V5的最晚发生时间 7 减去活动 a4 所需时间 1，即为6.

ETV	0	6	4	5	7	7	16	14	18
LTV	0	6	6	8	7	10	16	14	18
ETE	0	0	0	6					
LTE	0	2	3	6					

顶点	顶点	权值
V1	V2	6
V2	V5	1

第三步：访问顶点V3，遍历V3的邻接顶点V5 . 将活动a5的最早发生时间更新为事件V3的最早发生时间 4，将活动a5的最晚发生时间更新为事件V5的最晚发生时间 7 减去活动a5所需时间 1，即为6.

ETV	0	6	4	5	7	7	16	14	18	
LTV	0	6	6	8	7	10	16	14	18	
ETE	0	0	0	6	4					
LTE	0	2	3	6	6					

边数

顶点	顶点	权值
V1	V2	6
V2	V5	1

第四步：访问顶点V4，遍历该顶点的邻接顶点V6，将活动a6的最早发生时间更新为事件V4的最早发生时间5，活动a6的最晚发生时间为事件V6的最晚发生时间10 减去 活动 a6 所需要的时间 2，即为8.

ETV	0	6	4	5	7	7	16	14	18	
LTV	0	6	6	8	7	10	16	14	18	
ETE	0	0	0	6	4	5				
LTE	0	2	3	6	6	8				

边数

顶点	顶点	权值
V1	V2	6
V2	V5	1

第五步：访问顶点 V5，并分别遍历该顶点的邻接顶点V7 和顶点 V8。将活动a7和a8的最早发生时间更新为时间V5的最早发生时间7，最晚发生时间也更新为7。

ETV	0	6	4	5	7	7	16	14	18	
LTV	0	6	6	8	7	10	16	14	18	
ETE	0	0	0	6	4	5	7	7		
LTE	0	2	3	6	6	8	7	7		

边数

顶点	顶点	权值
V1	V2	6
V2	V5	1
V5	V7	9
V5	V8	7

第六步：访问顶点V6，并遍历该顶点的邻接顶点 V8，将活动a9的最早发生时间更新为事件V6 的最早发生时间7，最晚发生时间更新为事件V8的最晚发生时间 14 减去活动a9所需时间4，即为10。

ETV	0	6	4	5	7	7	16	14	18		
LTV	0	6	6	8	7	10	16	14	18		
ETE	0	0	0	6	4	5	7	7	7		
LTE	0	2	3	6	6	8	7	7	10		

顶点	顶点	权值
V1	V2	6
V2	V5	1
V5	V7	9
V5	V8	7

边数

第七步：访问顶点 V7，更新活动 a10的最早发生时间和最晚发生时间。

ETV	0	6	4	5	7	7	16	14	18		
LTV	0	6	6	8	7	10	16	14	18		
ETE	0	0	0	6	4	5	7	7	7	16	
LTE	0	2	3	6	6	8	7	7	10	16	

顶点	顶点	权值
V1	V2	6
V2	V5	1
V5	V7	9
V5	V8	7
V7	V9	2

边数

第八步：访问顶点V8，更新活动 a11的最早发生时间和最晚发生时间。

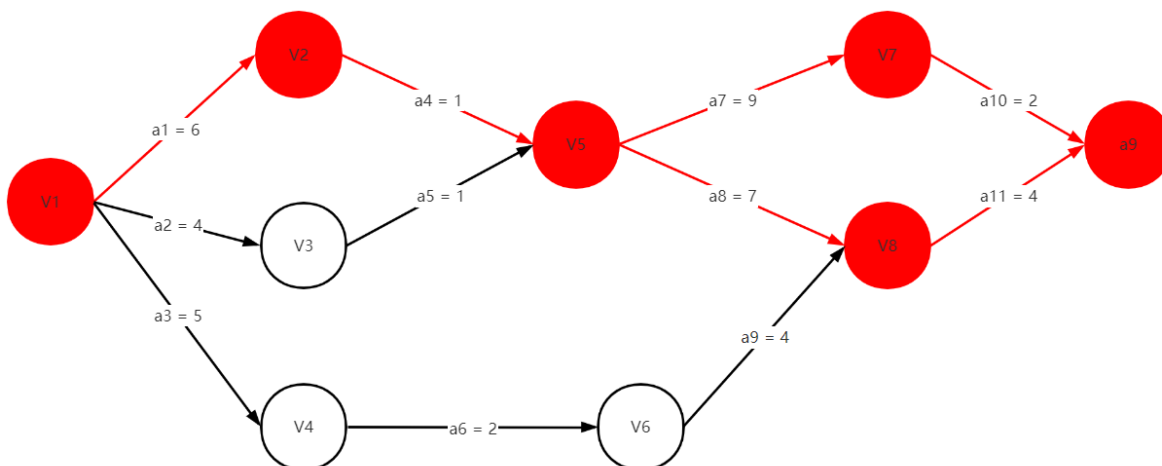
ETV	0	6	4	5	7	7	16	14	18		
LTV	0	6	6	8	7	10	16	14	18		
ETE	0	0	0	6	4	5	7	7	7	16	14
LTE	0	2	3	6	6	8	7	7	10	16	14

顶点	顶点	权值
V1	V2	6
V2	V5	1
V5	V7	9
V5	V8	7
V7	V9	2
v8	v9	4

边数

第九步：访问顶点V9，没有邻接顶点，什么都不做。

这样我们获得一个带权有向无环图中的关键路径：



应用与总结

(1) 完成整个工程至少需要多少时间；

(2) 哪些活动是影响工程的关键。

1956年，美国杜邦公司提出关键路径法，并于1957年首先用于1000万美元化工厂建设，工期比原计划缩短了4个月。杜邦公司在采用关键路径法的一年中，节省了100万美元。

说的更现实一点儿，以后大家如果要考PMP，那更是必须学会的算法了，有一天你成为了一个项目组组长，或者一个公司的老总，或者一个工程师，那么你就知道如何安排工程，知道什么是重点要关注的，知道如何进行资源配置。

向关键路径要时间，向非关键路径要资源

项目关键工作和关键路径的确定帮助项目经理抓住了项目工期管理中的主要矛盾，对于关键活动，优先安排资源，挖掘潜力，采取相应措施，尽量压缩需要的时间。而对非关键路径的各个活动，只要在不影响工程完工时间的条件下，抽出适当的人力、物力和财力等资源，用在关键路径上，以达到缩短工程工期，合理利用资源等目的。

对不可控的任务，可采取办法规避安排在关键路径上面

关键路径要提前发现风险，前期采取必要的措施规避掉

关键路径是动态的，非关键路径有可能变成关键路径，所以项目过程中也要随时注意非关键路径的状况

关键路径的查找过程中主要涉及四个概念，事件（顶点）的最早发生时间ETV，事件（顶点）的最晚发生时间LTV，活动（弧）的最早发生时间ETE，活动（弧）的最晚发生时间为LTE，其中事件的最早发生时间通过拓扑排序获得，而事件的最晚发生时间是通过逆向访问顶点（从源点到汇点的方向）结合最早发生时间获得，弧的活动的最早与最晚发生时间则是通过遍历一遍所有的弧结合事件的最早与最晚发生时间获得。关键路径在实际工程中应用广泛，如果你懂了，可以提高工程的工作效率，也可以适当地偷懒。

